ScholarBee Recommendation Engine

Technical Documentation

A detailed technical breakdown of onboarding, behavioral, hybrid, and Bayesian scoring models.

1. Executive Summary

The ScholarBee Recommendation System is a personalized, state-of-the-art academic recommendation engine built inside a NestJS environment. It provides targeted program (degree courses) and university recommendations by blending demographic onboarding preferences, student behavior logs (such as page clicks, course favorites, and applications), and global popularity analytics.

To handle cold-start scenarios, user fatigue, and dynamic shifts in student interest, the engine relies on a **Beta-Binomial Bayesian Weight Update** model. Rather than keeping preference weights constant, the system actively updates user weights in real-time as users browse, click, and interact.

> **Design Priorities Met**
>
> - **No Hardcoded Filtering Noise:** Silent dimensions like unimplemented search alignment or dwell times are scored as 0 to maintain clean signals.
> - **MOU & Freshness Tiebreakers:** Multiplicative boosts promote partner universities and active deadlines without overriding academic match preferences.
> - **Fast-acting Hybrid Shifts:** The behavioral event volume threshold is set to 1 so that the user's very first click is immediately visible in their rankings.

2. Design Methodology Rationale

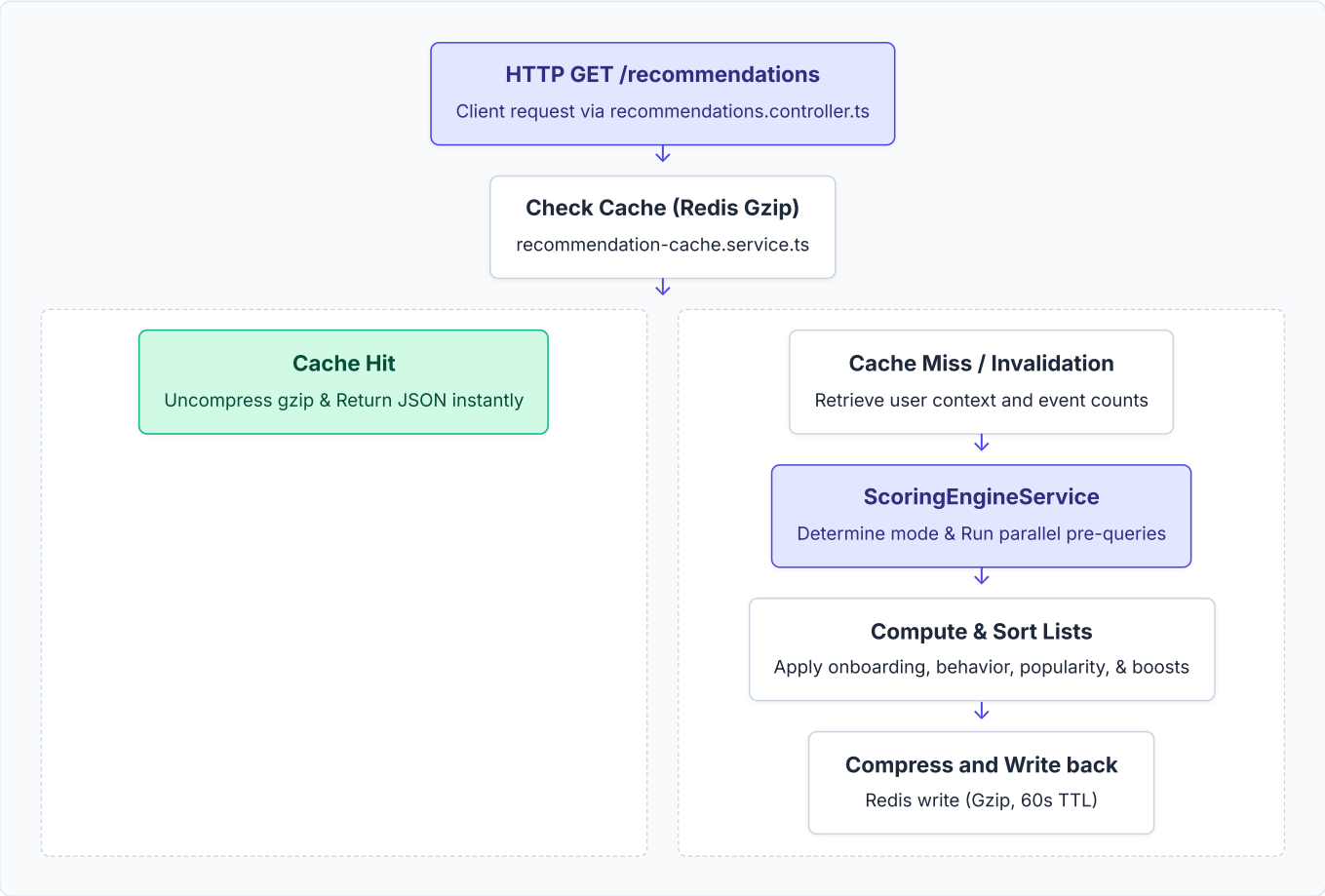
Building a recommendation engine for an academic portal introduces unique challenges. Traditional algorithms (like collaborative filtering or static search-based rules) suffer from cold-start limitations, lack real-time reactivity, or require high-overhead background processes. The ScholarBee engine resolves this by implementing an **Adaptive Bayesian Hybrid model**.

Why We Chose This Approach

- **Cold-Start Mitigation:** Pure behavioral systems fail when a student first registers. By leveraging demographic onboarding preferences, we establish high-quality baseline recommendations instantly.
- **Fast-Acting Reactivity:** Setting the behavioral activation threshold to `1` ensures the student's very first interaction (click, favorite, apply) dynamically shifts rankings, rewarding active exploration immediately.
- **Mathematical Stability:** Rather than using raw counter ratios which fluctuate wildly with a few clicks, the **Beta-Binomial Bayesian framework** acts as a natural stabilizer. Prior onboarding preferences provide a statistical "inertia" that prevents temporary browsing anomalies from completely skewing the recommendation profile.
- **Operational Simplicity & Speed:** Heavy machine learning frameworks (e.g., Matrix Factorization, Deep Learning) require dedicated GPU infrastructure, offline training, and complex pipelines. The Bayesian update equations are computationally lightweight, allowing them to execute inline inside the NestJS API thread, backed by gzipped Redis caching.

3. Architecture Overview

The recommendation engine resides inside the `recommendations/` module. It queries MongoDB for master data (Universities, Campuses, Addresses, Admissions, Programs, Program Templates) and uses Redis as a high-performance cache.



Query Performance Audits

To avoid high-overhead in-memory mapping and full-database table scans, the engine runs optimized Mongoose pipelines:

1. **Parallel Resolve Promises:** Resolves active admissions deadlines, preferred cities, and degree goals concurrently using `Promise.all` before fetching target programs.
2. **Read-only Lean Executions:** Executes `.lean()` database queries, bypassing full Mongoose document class hydration.
3. **Explicit Projections:** Projections filter out heavy fields (e.g. descriptions, gallery links, photos), querying only references needed for matching.

4. Data Models

Onboarding Preferences

Demographic and academic parameters collected from the user during signup or onboarding:

```
interface OnboardingPreferences {
  degree_goal: string;              // e.g. "Bachelors", "Masters", "PhD"
  preferred_cities: string[];        // e.g. ["Lahore", "Islamabad"]
  preferred_fields_of_study: string[]; // e.g. ["Computer Science", "Finance"]
  semester_fee_range: {
    min: number;
    max: number;
  };
  previous_marks_range?: {
    min_percent: number;
    max_percent: number;
  };
}
```

Bayesian Weights Schema

Stored directly inside the Student document (under `bayesian_weights`) to track preference relevance:

```
interface BayesianWeightDimension {
  alpha: number; // Positive reinforcement count (successes)
  beta: number;  // Negative/neutral interaction count (failures)
}

interface WeightPriors {
  degree_match: BayesianWeightDimension;
  field_match: BayesianWeightDimension;
  city_match: BayesianWeightDimension;
  fee_match: BayesianWeightDimension;
}
```

User Interaction Events Schema

Monitors student clicks, favorites, and applications:

```
const UserEventSchema = new Schema({
  user_id: { type: Schema.Types.ObjectId, ref: 'User' },
  session_id: { type: String },
  event_type: { type: String, enum: ['click', 'favorite', 'apply', 'ignore'] },
  resource_type: { type: String, enum: ['admission_program', 'university'] },
  resource_id: { type: Schema.Types.ObjectId },
  // Decayed/Extracted properties for quick matching
  field: { type: String },
  city: { type: String },
  degree_level: { type: String },
  fee_bucket: { type: Number }, // 1: <100k, 2: 100-200k, 3: 200-300k, 4: >300k
  created_at: { type: Date, default: Date.now }
});
```

5. API Endpoints

1. Fetch Recommendations

Retrieves paginated recommendations for programs or universities.

```
GET /api/v1/recommendations?type=programs&page=1&limit=10
GET /api/v1/recommendations?type=universities&page=1&limit=10

Headers:
Authorization: Bearer <token>
```

2. Track User Interaction Event

Tracks user behavior, invalidates relevant Redis caches, and recalculates Bayesian priors.

```
POST /api/v1/recommendations/events
Payload:
{
  "event_type": "click", // click, favorite, apply, ignore
  "resource_type": "admission_program", // admission_program, university
  "resource_id": "6a1ca58b520cbc85dc51d387",
  "metadata": {
    "source": "search_results"
  }
}
```

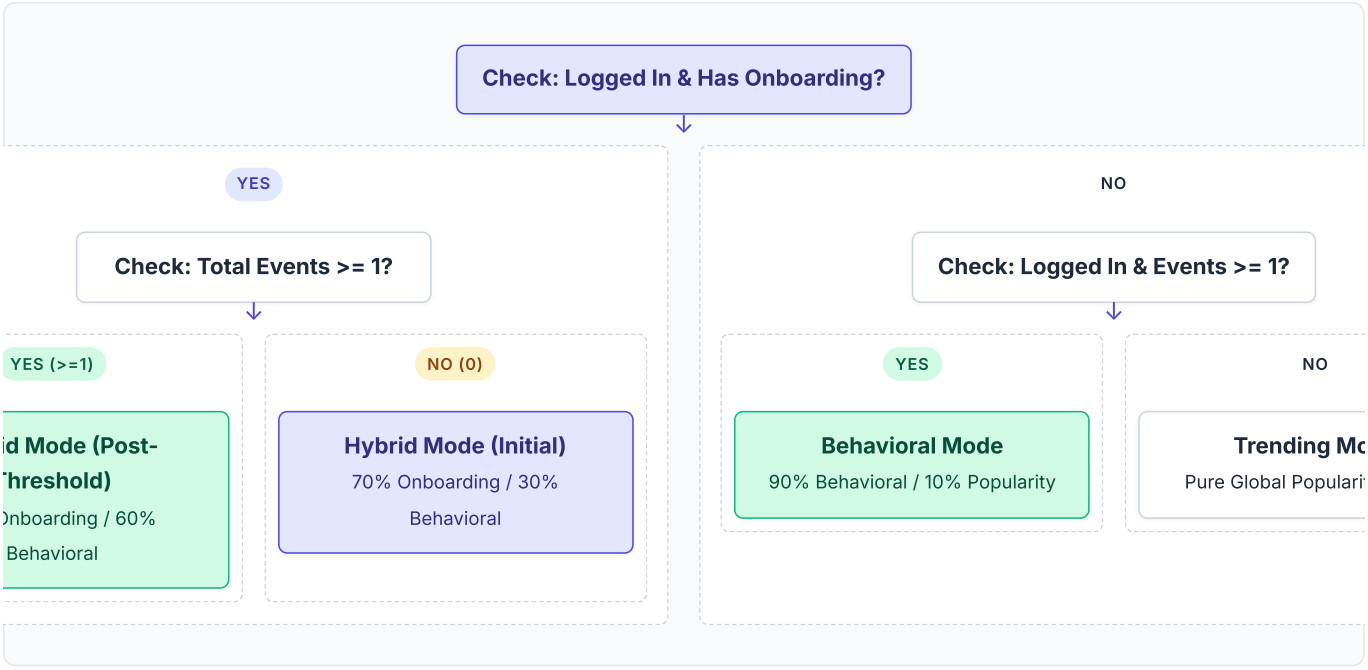
3. Seed Recommendation Demo

Utility endpoint to quickly seed onboarding preferences and simulated clicks for a demo user.

```
POST /api/v1/recommendations/seed-demo
```

6. Scoring Mode Decision Tree

The engine identifies one of four scoring modes based on the availability of onboarding preferences and the volume of behavioral event history.



Note on Program vs University mode differentiation: For program recommendations, if the user has onboarding preferences and 0 interaction logs, the scoring mode is set directly to `hybrid` (with alpha=0.7 and beta=0.3) so that the first click immediately affects rankings. For university scoring, a cold-start preference-only user matches `onboarding` mode.

7. Program Scoring Engine

Programs are scored out of a baseline range of [0.0, 1.0], scaled by boosts.

```
Final Score = [(1.0 - PopWeight) * ( α * OnboardingScore + β * BehavioralScore ) + PopWeight *
                        ProgramPopularity] * MOUBoost * FreshnessBoost
```

Score Formulas Breakdown

1. Onboarding Score (Max 1.0)

Sums matched preference parameters. The dimensions are weighted based on Bayesian priors (which default to 25/25/20/15 proportions scaled to sum to 0.85):

```
OnboardingScore = (wDegree * MatchDegree) + (wField * FieldSim) + (wCity * MatchCity) + (wFee * MatchFee) +
                            0.10 * MatchEligibility + 0.05 * MatchTimeline
```

- `MatchDegree`: Binary match (1.0 if program's degree level fits, 0.0 otherwise).
- `FieldSim`: Numerical score (0.0 to 1.0) returned from the Knowledge Graph similarity mapping.
- `MatchCity`: Binary match (1.0 if program campus city matches user preferred cities).
- `MatchFee`: Dynamic budget score (1.0 if program fee is within user's preferred min/max range, 0.5 if up to 20% over range, 0.0 if exceeded).
- `MatchEligibility / MatchTimeline`: Fixed values set to 1.0 placeholder matches until dynamic indicators are implemented.

2. Behavioral Score (Max 1.0)

Weighted sum of individual interaction signals:

```
BehavioralScore = 0.20 * (Clicks / MaxClicks) + 0.30 * MatchApplies + 0.20 * Favorited + 0.15 * SearchAlign
                                          + 0.15 * DwellTime
```

- `Clicks / MaxClicks`: Decayed interaction clicks relative to the user's most clicked program. Clicks decay exponentially over a 30-day half-life.
- `MatchApplies`: Binary (1.0 if applied, 0.0 otherwise).
- `Favorited`: Decayed bookmark status (1.0 if favorited, 0.0 otherwise).
- `SearchAlign / DwellTime`: Hardcoded to 0 to prevent noise.

3. Program Popularity (Capped at 1.0)

Evaluates active popularity inside the trending logs:

```
ProgramPopularity = Min( [ FavoritesCount + RedirectsCount + DecayedGlobalClicks ] / 10.0, 1.0 )
```

Global clicks include a 5x weight multiplier for application actions (`APPLICATION_EVENT_POPULARITY_WEIGHT = 5.0`).

4. Multiplicative Boosts

- **MOU Boost (1.03):** ScholarBee verified campuses receive a 3% boost. It serves as a tiebreaker for programs with similar academic fit.
- **Freshness Boost (1.02):** Programs with active deadlines receive a 2% boost to promote timely applications.

8. University Scoring Engine

University scoring follows a simplified structure since Bayesian updates are targeted exclusively to program-related choices.

Score Formulas Breakdown

1. Onboarding Score (Max 1.0)

A direct, non-Bayesian evaluation based on location and degree alignments:

```
OnboardingScore = 0.40 * MatchCity + 0.30 * MaxFieldSimilarity + 0.30 * Baseline
```

- `MatchCity`: 1.0 if the university has a campus in one of the user's preferred cities, 0.0 otherwise.
- `MaxFieldSimilarity`: Highest similarity score (0.0 to 1.0) between any preferred field of study and any course template offered by the university.
- `Baseline`: A static 0.30 baseline score applied to all universities.

2. Behavioral Score (Max 1.0)

Reflects interactions with the university listing page or its associated programs:

```
BehavioralScore = 0.50 * Clicks + 0.50 * Applies
```

3. University Popularity (Capped at 1.0)

```
UniversityPopularity = Min( DecayedGlobalClicks / 10.0, 1.0 )
```

9. Bayesian Weight System

ScholarBee uses a **Beta-Binomial Bayesian framework** to continuously update onboarding preference weights.

Mathematical Rationale

For each preference dimension (Degree, Field, City, Fee), we represent the user's focus on that dimension as a Beta probability distribution:

$$\theta_d \sim \text{Beta}(\alpha_d, \beta_d)$$

Where `α` represents successes (clicking programs matching the preference) and `β` represents failures/indifference (clicking programs that do not match the preference).

Weight Update Rule

On every user event tracked, we check if the interacted program matches the user's preferences:

User Interaction (Action Strength)	Match Result	Update Applied
Positive Interaction: • Apply (strength = 3) • Favorite (strength = 2) • Click (strength = 1)	Matches preference dimension (Match = 1)	`α += 1.0 * strength`
	Does not match preference (Match = 0)	`β += 1.0 * strength`
Negative Interaction: • Ignore (strength = 0.5)	Matches preference dimension (Match = 1)	`β += 1.0 * strength`
	Does not match preference (Match = 0)	`α += 1.0 * strength`

Weight Normalization

When retrieving weights, the engine computes the mathematical mean of each Beta distribution (`α / (α + β)`), and then normalizes them so they sum to 1.0. This normalized value is multiplied by 0.85 to leave 15% reserved for fixed criteria:

$$\text{Mean}_d = \alpha_d \; / \; (\; \alpha_d + \beta_d \;)$$

$$\text{wDegree} = [\; \text{Mean}_{\text{degree}} \; / \; \Sigma(\text{Means}) \;] * 0.85$$

10. Knowledge Graph & Taxonomy

The Knowledge Graph performs string cleanup, maps spelling variants to canonical categories, and calculates cross-disciplinary similarities.

1. String Normalization

Cleans field inputs by removing dots, brackets, hyphens, and multi-spaces, converting abbreviations (e.g. `BS CS`, `BSCS`) to a canonical snake_case key (`computer_science`).

2. 2-Hop Similarity Propagation

Direct taxonomic links are pre-seeded in code (e.g., Computer Science to Software Engineering = 0.90). To capture secondary relationships, the graph runs a **two-hop transitive mapping** with a 30% penalty (0.7 coefficient):

```
Sim(A, C) = Max( DirectSim(A, C), DirectSim(A, B) * DirectSim(B, C) * 0.7 )
```

Example Path:

- `Artificial Intelligence` ↔ `Computer Science` (Direct = 0.85)
- `Computer Science ↔ Electrical Engineering` (Direct = 0.60)
- Computed 2-Hop link: `Artificial Intelligence` ↔ `Electrical Engineering` = `0.85 * 0.60 * 0.7 = 0.357`

11. Caching Layer & Performance Optimization

To prevent heavy database queries on high-traffic pages, recommendation lists are aggressively cached.

Cache Compression GZIP

Recommendation JSON responses are serialized and compressed using `zlib.gzip` before storage in Redis, reducing payload sizes by up to 90%.

Cache TTL 60 SECONDS

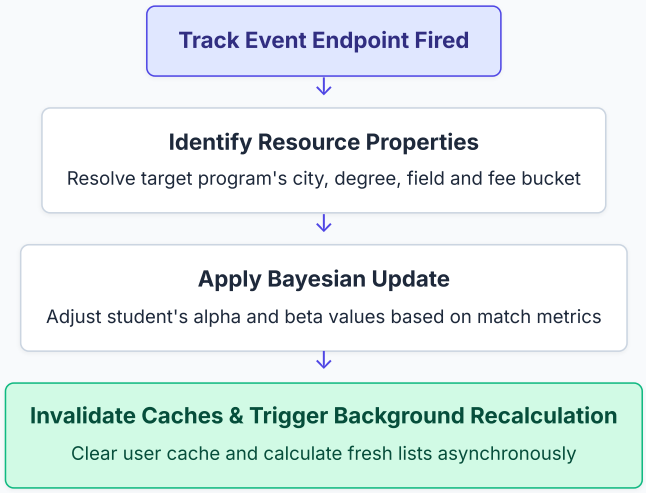
Cached lists expire after 60 seconds. This maintains responsiveness for trending program statistics while ensuring database reads stay minimal.

Invalidation REACTIVE

Any tracked user event immediately invalidates that user's Redis caches and triggers a background computation.

12. Event Tracking Flow

When a user event is logged:



13. Recommendation Score Simulator Structure

The recommendation engine has a companion simulation logic matching the formulas implemented in `scoring-engine.service.ts`. The simulator accepts input attributes for the student, course, and interactions, computing the final relevance score using the exact weights and decay models.

Simulator Mathematical Validation Structure

The simulation runs three independent pipelines representing User Preferences, Target Course Attributes, and Interaction Histories to compute the final relevance score.

1. User Onboarding Preferences

Degree Goal: Bachelors

Preferred Field: Computer Science

Preferred Cities: Lahore

Semester Budget: Up to 300,000 PKR

2. Target Course Attributes

Degree Level: Bachelors

Field of Study: Computer Science

Campus City: Lahore

Tuition Fee: 250,000 PKR

Verified Partner: Yes (MOU Boost 1.03x)

Active Deadline: Yes (Freshness Boost 1.02x)

3. Interactions & Popularity

User Clicks: 0 Clicks

User Applied: No

User Favorited: No

Global Decayed Clicks: 18 clicks

Total Interaction Logs: 0 (Hybrid Initial Mode)

0.767

FINAL RELEVANCE SCORE

Calculated Mode: Hybrid Mode (Initial: 70% Preference / 30% Behavioral)
Onboarding Preference Score: 1.000 (Exact match)
Behavioral Component Score: 0.000 (No history)
Decayed Popularity Score: 1.000 (18 global clicks)
Multipliers: MOU Boost (1.03) * Freshness Boost (1.02)

14. Configuration Reference Table

The scoring engine utilizes values exported from `scoring.constants.ts`.

Constant Name	Default Value	Type	Logic & Tuning Rationale
`INITIAL_ONBOARDING_WEIGHT_ALPHA`	`0.7`	Number	Priority weight for user demographic preferences when the interaction history is low. Ensures high accuracy early on.
`INITIAL_BEHAVIORAL_WEIGHT_BETA`	`0.3`	Number	Behavioral priority weight during cold-start. High enough to surface immediate clicks, but low enough not to overshadow preference matches.
`POST_THRESHOLD_ONBOARDING_WEIGHT_ALPHA`	`0.4`	Number	Onboarding weight after the student has browsed enough. Shifts focus towards behavioral interests.
`POST_THRESHOLD_BEHAVIORAL_WEIGHT_BETA`	`0.6`	Number	Behavioral weight after the student has browsed enough. Interaction history now takes precedence.
`BEHAVIORAL_EVENT_THRESHOLD`	`1`	Number	The event volume threshold required to transition weights from Initial to Post-Threshold values. Set to 1 for quick adaptation.
`MOU_BOOST`	`1.03`	Number	Verified partner campuses receive a 3% boost. Operates as a tiebreaker for similar ranking matches.
`FRESHNESS_BOOST`	`1.02`	Number	Active application deadlines receive a 2% boost to keep recommended items active.
`HALF_LIFE_DAYS`	`30`	Number	Days after which a user interaction's score decays by half. Prevents outdated clicks from skewing current preferences.
`ONBOARDING_POPULARITY_WEIGHT`	`0.2`	Number	Percentage of final score determined by global program popularity in onboarding mode. Sets popular options as default.
`APPLICATION_EVENT_POPULARITY_WEIGHT`	`5.0`	Number	Multiplying factor for application events relative to clicks. Strongly weighs courses that students actually apply to.